

Timed Traffic Controller

Observe a request, decide by time, show the state

The lamps are both the circuit's output and its live explanation.

Lesson	011 — deterministic behavior	Time	60–90 minutes
Prerequisites	Lessons 001–010	Board	Arduino Mega 2560
Build	eight signal LEDs, one button, built-in D13	Run	about two minutes
Safety	E1: USB-powered, resistor-limited LEDs only		

1 Mission and evidence chain

Build the nonblocking controller that lesson 012 will compose into a tabletop junction. Its evidence chain is:

D22 press → debounced event → latched request → timed phase → D30–D37 lamps.

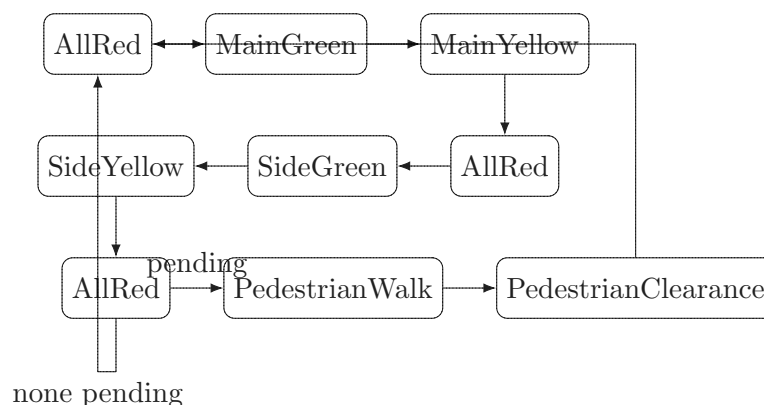
The D13 ready LED separately reports successful resource acquisition. It does not claim that the signal policy is correct. The eight signal LEDs expose the policy while it runs; a meter at a named lamp pin proves the electrical level.

By the end, you can:

- distinguish elapsed-time decisions from blocking delays;
- supply all time and input events explicitly to `TrafficJunction`;
- explain request latching and the release-gated `Button`;
- predict the next phase from the lamps;
- make a button press wait for a safe walk phase;
- explain why conflicting greens never appear.

Safety checkpoint. Disconnect USB before wiring. Every LED needs its own 330Ω or larger series resistor. Verify LED polarity and the breadboard's internal connections unpowered. Stop for heat, odor, resets, unstable USB power, or a signal combination that differs from the truth table. This is a low-voltage learning model, not a road-signal controller.

2 The state machine is the design



The first `update()` anchors startup `AllRed`; it does not borrow time from construction or from Arduino's global clock. Every later transition occurs when the supplied `TimePoint` reaches the exact deadline. A press event becomes a pending request. Startup may serve it immediately after startup `all-red`; otherwise the controller retains it until the next safe pedestrian window after the side-road sequence.

Phase	Lamps that are on	Invariant
AllRed	main red, side red, pedestrian stop	no vehicle green
MainGreen	main green, side red, pedestrian stop	one vehicle green
MainYellow	main yellow, side red, pedestrian stop	no vehicle green
SideGreen	main red, side green, pedestrian stop	one vehicle green
SideYellow	main red, side yellow, pedestrian stop	no vehicle green
PedestrianWalk	both vehicle reds, pedestrian walk	vehicles held red
PedestrianClearance	both vehicle reds, pedestrian stop	vehicles held red
Fault	both vehicle reds, pedestrian stop	latched until explicit reset

3 Exact build drawing

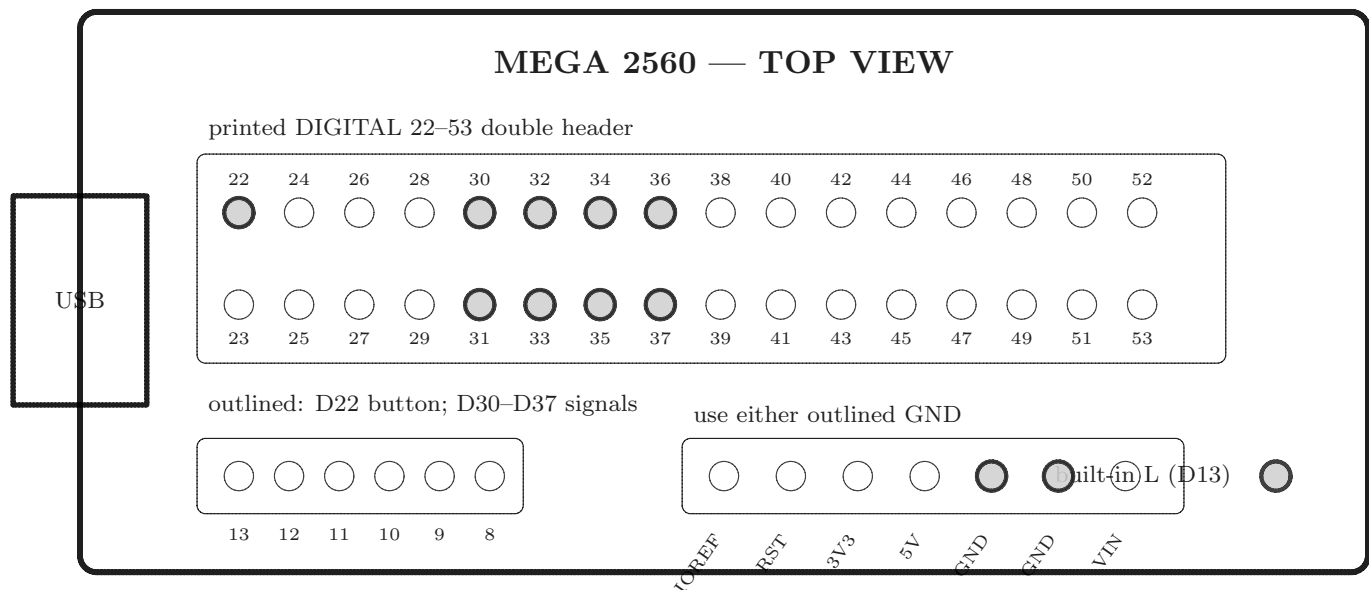
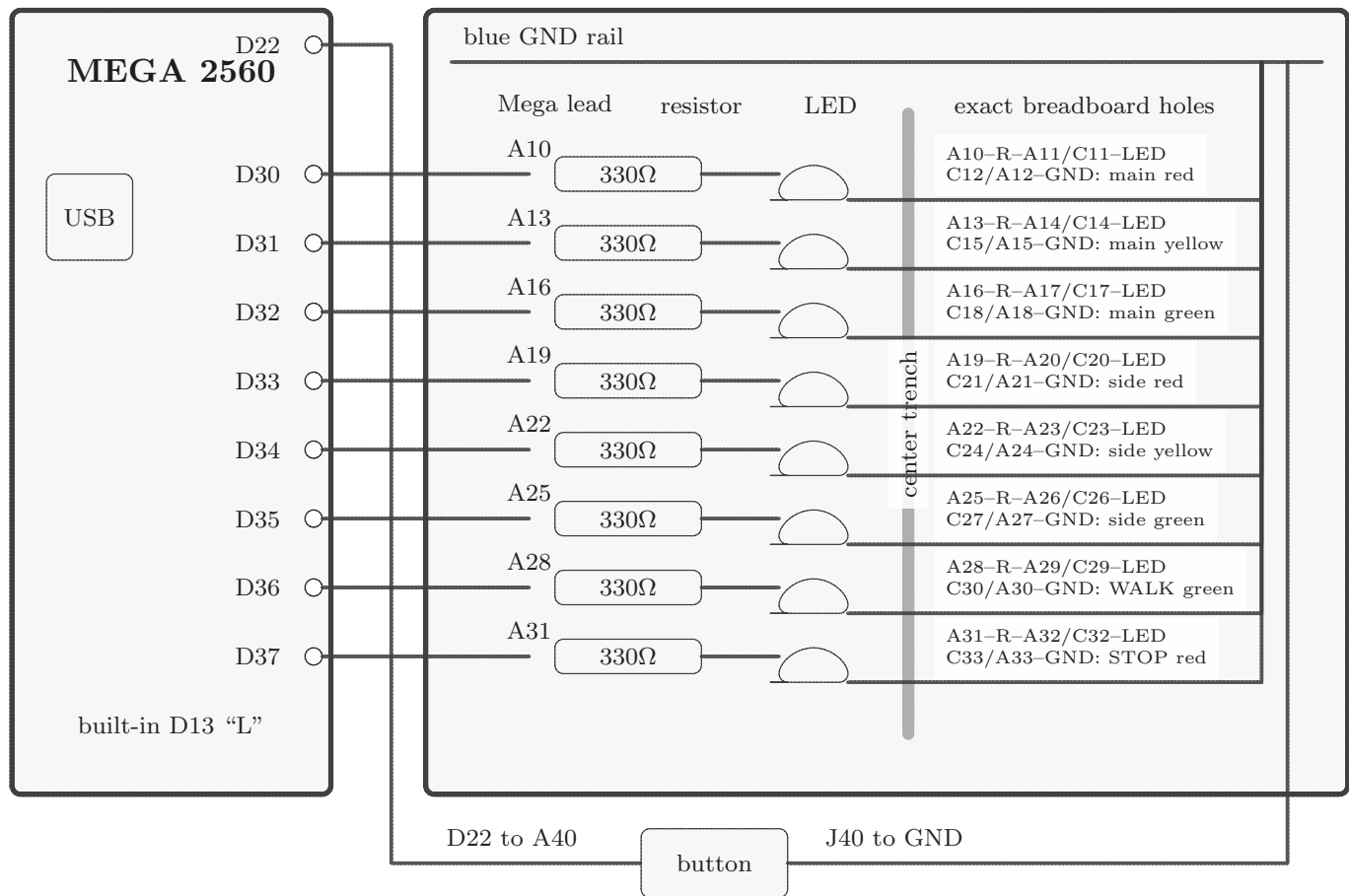


Figure 1: Physical Mega header locator, top view with USB at left. Read the board’s printed numbers before inserting each lead: D30–D37 alternate between the two rows of the long double header. D13 is the built-in acquisition light; use either outlined power-header GND socket.



Button straddles trench: legs E40/E42 and F40/F42.

Figure 2: Authoritative coordinate layout. Each LED consumes three different breadboard rows: Mega lead at A_n , resistor $A_n - A_{n+1}$, LED $C_n + 1 - C_{n+2}$, then A_{n+2} to GND. The button coordinates identify all four legs. A–E holes with the same row number are one common strip; F–J are the separate strip across the trench. Match these labels before applying USB power.

4 Parts and literal wiring

Quantity	Part	Requirement
1	Mega 2560	USB powered
3	red LEDs	main red, side red, pedestrian stop
2	yellow LEDs	identified polarity and forward-voltage range
3	green LEDs	main green, side green, pedestrian walk
8	resistors	330Ω or larger, one per external LED
1	momentary button	normally open, used with internal pull-up
1	multimeter	DC voltage and unpowered continuity

Every LED path: named Mega pin → 330Ω → LED anode; LED cathode → GND. D22 connects through the normally open button to GND; Pull::Up supplies its idle level.

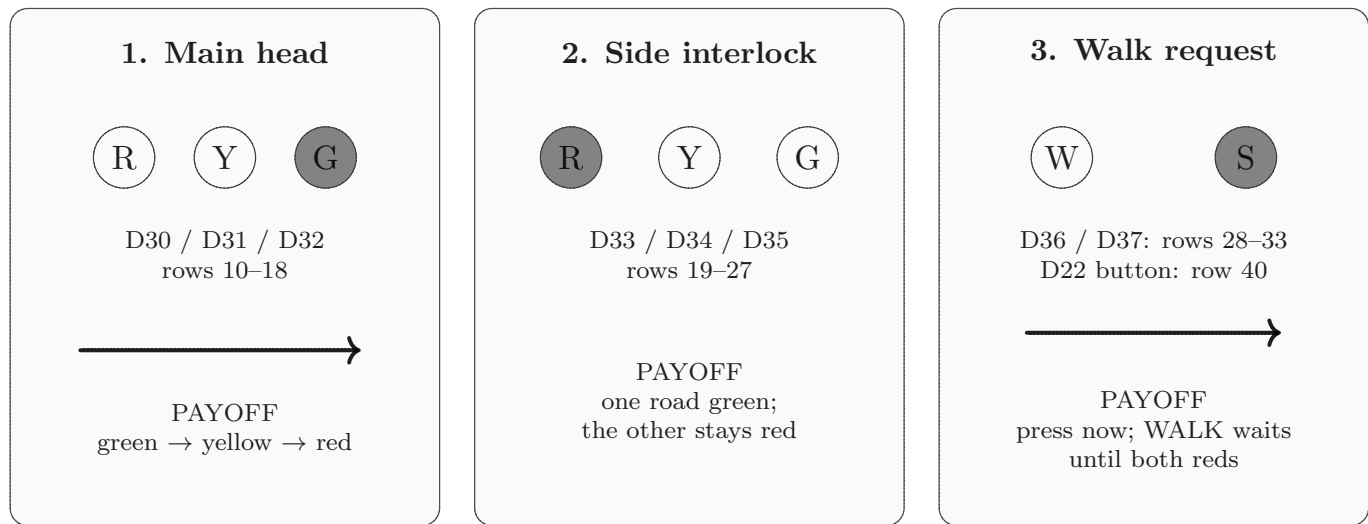


Figure 3: Three visible wins. Each plate names only the new pins and the behavior that becomes possible after that stage is wired correctly.

Function	Pin	Circuit-native observation
main red / yellow / green	D30 / D31 / D32	main signal head
side red / yellow / green	D33 / D34 / D35	side signal head
pedestrian walk / stop	D36 / D37	pedestrian signal head
request button	D22 to GND	press event; pending state appears later as walk
resource ready	built-in D13 “L” LED	250 ms pulse after acquisition
electrical test point	selected lamp pin to GND	near 0 V off; near 5 V on

With a conservative 2.0 V LED forward voltage, a 330Ω resistor permits about $(5 - 2)/330 = 9.1$ mA. The normal truth table lights three external LEDs at most: about 27.3 mA total LED current, with no pin above 9.1 mA. Use identified ordinary indicator LEDs whose stated continuous current permits that value. Power this build only from the Mega’s USB connector; do not attach an external supply.

5 Read the public model

```

adk::TrafficConfig config;
adk::TrafficJunction traffic (config);

traffic.initialize ();
traffic.update (now, adk::TrafficInput (requestEvent, circuitHealthy));

const adk::TrafficSnapshot decision = traffic.snapshot ();

traffic.reset    ();
traffic.shutdown ();

```

TrafficConfig names each duration: **startupAllRed**, **vehicleAllRed**, **mainGreen**, **mainYellow**, **sideGreen**, **sideYellow**, **pedestrianWalk**, and **pedestrianClearance**. Defaults are 1,000; 500; 5,000; 1,500; 4,000; 1,500; 5,000; and 2,000 ms respectively.

TrafficSnapshot exposes phase and status, output signals, phase start and next deadline, request state, change and acceptance events, deadline presence, and transition count:

```

phase, status, signals,
phaseSince, nextDeadline,
pedestrianRequestPending,
phaseChanged, requestAccepted, hasDeadline,
transitionCount

```

Observation does not consume an event. Supplying the same configuration, timestamps, and input events produces the same snapshots. Repeated `initialize()` is idempotent: it preserves the active phase, timing, pending request, or fault latch. `reset()` explicitly starts a new run in startup **AllRed**; it requires an initialized object. `shutdown()` is idempotent, leaves an inert all-red snapshot, and makes later `update()` calls report `NotInitialized`.

The canonical sketch uses the same high-level language as the circuit:

```

const adk::TrafficInput observation = observeRequest (now);
const adk::Status      decision     = decideSignals (now, observation);
const bool              signalsShown = showSignals   (traffic.snapshot ().signals);

if (!signalsShown)
{
    stopSafely ();
}

if (!decision.ok ())
{
    ready.off ();
}

```

There is no `delay()`. `loop()` remains free to debounce the button and actuate the current signal during every phase. The complete, canonical source is `Lesson011TimedTraffic`.

6 Build the show in four quick wins

- 1. Make the main road chase.** With USB disconnected, wire rows 10–18 and the GND rail exactly as drawn. The first reward is a recognizable main green, main yellow, all-red rhythm. If a lamp is dark, turn USB off and flip only that LED; the code cannot make the next pattern convincing while one path is wrong.
- 2. Add the side road.** Wire rows 19–27. Now the green permission moves from main to side only after yellow and all-red. Watch the two green LEDs: they must never overlap. This interlock is the point of the stage, not a form to fill out.
- 3. Add the tiny pedestrian signal.** Wire rows 28–33. STOP stays on during vehicle movement. WALK can appear only while both vehicle reds are on. The three-head picture therefore explains whether the wiring and phase logic agree.
- 4. Give it a request button.** Place the button across the breadboard center trench and connect D22 to one side and GND to the opposite side. Press it during main green. Nothing jumps dangerously: the current vehicle sequence finishes, both reds appear, and then WALK gets its turn. Try a second press; the release-gated button creates one event per deliberate click.

Build, then upload:

```

make arduino-Lesson011TimedTraffic
make upload-Lesson011TimedTraffic PORT=/dev/ttyACMO

```

D13 flashes for 250 ms when all resources are acquired, stays dark for 750 ms, and then the signals begin.

After two minutes the controller shows all-red for one second and releases every pin. Press Mega RESET to play again; remove USB before moving any wire.

The satisfying moment: press during main green, call out the remaining phases, and watch WALK arrive exactly where you predicted. Because WALK depends on both vehicle reds and the button event, the payoff only appears when the whole chain is working.

7 If the picture is wrong

What you see	Fastest useful move
D13 never flashes	remove USB; compare the exact pin labels and GND rail
one lamp never lights	remove USB; flip that LED and inspect its 330 Ω path
wrong lamp lights	move that row's Mega lead to the pin printed in the drawing
button does nothing	use opposite button legs across the center trench
both greens appear	remove USB immediately and recheck D30–D37 row by row

The library's timing, rollover, fault, and resource-release checks remain in the automated host suite. They protect this playful build without turning the lesson into a test worksheet.